



Shifa Tameer-e-Millat University^{TY}
شفّا تعميرِ ملّت یونیورسٹی

Software Requirement Specification (SRS DOCUMENT)

for

Professor OS

Version 1.1

By

Muhammad Yasin Asif BSSE-23S-0036

Muhammad Fahim BSSE-23S-0033

Rayyan Shahid BSSE-23S-0037

Supervisor

Ms. Habib-un-Nisa

Co-Supervisor

Mr. Muhammad Shaban Qabil

Bachelor of Science in Software Engineering (2023-2027)

Department of Software Engineering

Faculty of Computing

Revision History

Name	Date	Reason for Changes	Version
SRS Document	May 2026	Initial SRS submission all modules, FRs, NFRs, RTM complete	1.0
SRS Document	June28, 2026	Standardized identifier formatting; integrated React frontend visualization library (SI-8); decomposed consolidated Use Case diagrams into actor-specific flows for improved clarity.	1.1

Application Evaluation History

Comments (by committee) *include the ones given at scope time both in doc and presentation	Action Taken
Evaluators noted excessive system complexity and highlighted that consolidated Use Case diagrams were cluttered, unreadable, and failed to adequately isolate actor-specific system interactions.	Streamlined overall system scope; decomposed all consolidated Use Case diagrams into individual, actor-specific flows to improve visual clarity and architectural readability.

Supervised by

Ms. Habib-un-Nisa

Co Supervisor

Mr. Muhammad Shaban Qabil

Signature_____

Signature_____

Table of Contents

Contents

1. Introduction.....	6
1.1 Purpose	6
1.2 Scope.....	7
1.3 Modules	7
1.1 Business Objectives	8
2. Overall Description.....	9
2.1 Product Perspective	9
2.2 User classes and characteristics	10
2.3 Operating Environment	11
3. Requirement Identifying Technique	12
3.1 Observations	12
3.2 Story Boarding.....	12
4. Functional Requirements	12
4.1 Functional Requirement 1: Authentication and User Management (M-01).....	13
4.1.1 System Functional Requirements	13
FR-M01 – Authentication and User Management.....	13
FR-M04 – Explainable AI (XAI) Processing	15
FR-M06 – Analytics Dashboard.....	17
FR-M07 – Smart Proctoring	18
FR-M09 – AI Teaching Assistant.....	19
4.2 Use Case Modeling.....	20

4.3 Use Case Descriptions and Diagrams.....	20
4.4 Domain Model.....	38
5. Non-Functional Requirements.....	41
5.1 Selection of Quality Attributes	41
5.2 Reliability (NFR-R1).....	41
5.3 Performance (NFR-P1).....	42
5.4 Security (NFR-S1).....	42
5.5 Usability (NFR-U1).....	43
5.6 Maintainability (NFR-M1)	43
5.7 Scalability (NFR-SC1).....	44
5.8 Availability (NFR-A1).....	44
5.9 Compatibility (NFR-C1).....	44
6. External Interface Requirements	45
6.1 User Interfaces Requirements.....	45
6.2 Software interfaces	46
6.3 Hardware interfaces	47
6.4 Communications interfaces	47
7. Requirements Traceability Matrix.....	48
8. Storyboarding	51
9. References.....	56

1. Introduction

This Software Requirements Specification (SRS) documents the complete functional and non-functional requirements for ProfessorOS, an AI-powered academic assessment platform developed as a Final Year Project at Shifa Tameer-e-Millat University (STMU), Islamabad. The document serves as the primary reference for the development team, supervisors, and the FYP evaluation committee throughout the development cycle spanning April 2026 to February 2027.

1.1 Purpose

The purpose of this SRS is to clearly define the requirements of ProfessorOS so that everyone involved in the project has a shared understanding of what the system is expected to achieve. It identifies the services the platform must provide, the constraints under which it operates, and the quality attributes it must satisfy. This document is intended for the following stakeholders:

- **Development Team:**

Muhammad Yasin Asif (AI/ML, Backend, DevOps), Muhammad Fahim (Frontend, Analytics Dashboard), Rayyan Shahid (Mobile Application, Smart Proctoring, Code Editor)

- **Primary Supervisor:**

Ms. Habib-un-Nisa responsible for academic supervision, guidance, and approval of project milestones

- **Co-Supervisor:**

Mr. Muhammad Shaban Qabil responsible for technical review of the AI/ML architecture and system feasibility

- **FYP Evaluation Committee:**

Department of Software Engineering, STMU responsible for formal evaluation and grading of the project

- **End Users:**

University professors, teaching assistants, students, and administrative staff who will interact with the system

This document focuses on **what the system is required to do**. Implementation details, architectural decisions, algorithms, and technology-specific designs are documented separately to maintain a clear separation between requirements and design.

1.2 Scope

ProfessorOS is AI-powered academic assessment ecosystem designed for Pakistani universities. The platform automates the complete assessment lifecycle from course creation and rubric design through AI grading, explainable feedback, proctored examination, and performance analytics. All services run via a single Docker Compose deployment requiring no external cloud services or recurring costs.

The core problem it solves is the 12 to 18 hour per-week grading burden measured in a live observation at STMU, caused by manual marking of text essays, code submissions, and MCQ papers across cohorts of up to 180 students per assignment cycle. ProfessorOS targets a minimum 75 percent reduction in professor grading time, feedback delivery within 60 seconds of submission, inter-rater reliability above 85 percent, and a 60 percent reduction in single-device examination cheating through a locally running Llama 3.1 8B model with LIME and SHAP explainability.

The following capabilities are deliberately excluded from version 1.0: video proctoring, facial recognition, handwritten assignment scanning, direct ERP integration, and Urdu-language interface support.

1.3 Modules

ProfessorOS is organized into ten modules, each assigned a persistent label for traceability throughout this SRS:

Module ID	Module Name	Platform	Description
M-01	Authentication and User Management	Web + Mobile	JWT auth, biometric login, RBAC for all four user roles, bulk CSV import

M-02	Course and Assignment Management	Web (Professor)	Course CRUD, weighted rubric builder, assignment creation, TA delegation
M-03	AI Grading and Evaluation	Backend	Celery-queued Llama 3.1 8B rubric-anchored grading with human override
M-04	Explainable AI (XAI) Processing	Web	LIME token highlights and SHAP criterion attributions as colour overlays
M-05	Student Submission Portal	Web (Student)	Multi-type submission, XAI feedback viewer, grade dispute, notifications
M-06	Analytics Dashboard	Web (Professor)	Grade histogram, criterion radar chart, at-risk table, cohort trend chart
M-07	Smart Proctoring	Mobile (Flutter)	AppLifecycleState focus monitoring, violation logging, real-time WebSocket reporting
M-08	Code Editor and Compiler	Web + Mobile	Monaco Editor with Judge0 sandboxed compilation: C, C++, Python, Java, JavaScript, Go, Rust, PHP, Ruby, C#, TypeScript, Swift
M-9	AI Teaching Assistant	Web + Mobile	LangChain RAG pipeline over professor-uploaded materials for 24/7 Q&A

1.1 Business Objectives

The following business objectives (BO) drive the functional and non-functional requirements documented in this SRS. Each requirement is traceable to at least one BO in the Requirements Traceability Matrix (Appendix C).

ID	Business Objective
BO-1	Reduce professor grading time by a minimum of 75% compared to the current manual process measured at 12–18 hours per week at STMU.

BO-2	Deliver criterion-level AI feedback to students within 60 seconds of submission under a concurrent load of 180 or more students.
BO-3	Achieve inter-rater reliability above 85% across all AI-graded submissions using the same rubric and model version.
BO-4	Provide LIME and SHAP explainability overlays on every graded text submission so professors and students can see exactly which phrases drove each criterion score.
BO-5	Provide sandboxed in-browser code compilation and execution supporting 12 programming languages via a self-hosted Judge0 instance, eliminating professor manual code-setup overhead.
BO-6	Reduce single-device examination cheating by 60% through privacy-respecting app-switch detection without camera or facial recognition.
BO-7	Provide a 24/7 AI Teaching Assistant grounded in professor-uploaded course materials, reducing out-of-class student clarification delays.
BO-8	Provide real-time course analytics including grade distribution histograms, criterion radar charts, at-risk student identification, and cohort time-series charts.
BO-9	Deploy the complete platform via a single Docker Compose command with no external cloud dependencies, SaaS subscriptions, or recurring costs.

2. Overall Description

This section presents a high-level overview of ProfessorOS: its origin, the environment in which it will operate, the categories of users who will interact with it, and the constraints that apply.

2.1 Product Perspective

ProfessorOS is a new software system developed specifically for academic assessment. It is not an extension of an existing Learning Management System (LMS), although it is designed so that integration with institutional systems can be added in future releases.

The platform follows a modular architecture in which each major feature is implemented as an independent component. Examples include user management, course management, AI-assisted

grading, code execution, examination management, analytics, and notification services. This design allows new features to be added with minimal impact on existing modules.

The system follows a client-server architecture. Users interact with the application through a web browser or mobile application, while backend services manage authentication, business logic, AI processing, data storage, and communication with external services such as Judge0.

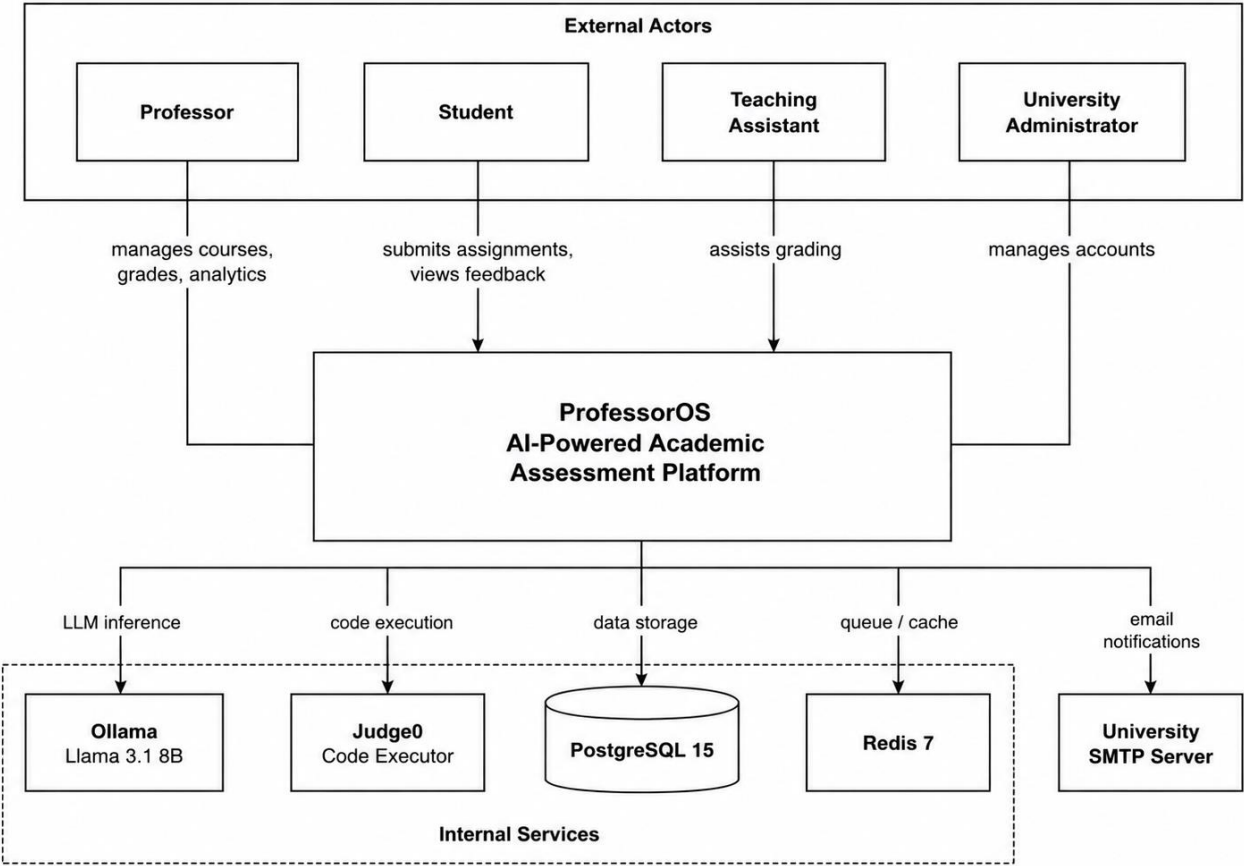


Figure 1: ProfessorOS — System Context Diagram

Figure 2.1.1: ProfessorOS System Architecture - Client-Server Overview

2.2User classes and characteristics

User Class	Description	Characteristics
------------	-------------	-----------------

Professor	Primary grader, rubric designer, and analytics consumer.	Technically moderate. Manages 2 courses with ~180 students each. Primary pain point: 12-18 hours/week grading burden.
Student	Submits assignments; views explainable feedback; uses mobile proctoring app.	Highly variable technical level. ~180 per course. Primary concern: 7-14 day feedback delays and opaque grading.
Teaching Assistant	Secondary grader assigned batches by the professor.	Familiar with course content and rubrics. Limited override authority. Volume: 1-3 per course.
University Administrator	Manages accounts and institutional configuration.	Non-technical. Needs bulk CSV import, system health dashboards, and institutional reporting.

2.3 Operating Environment

The following environment constraints apply to ProfessorOS:

ID	Operating Environment Constraint
OE-1	The system shall operate on university-owned server hardware running Ubuntu 22.04 LTS or later, with minimum 16 GB RAM. An NVIDIA GPU with at least 8 GB VRAM is recommended for Ollama LLM inference; CPU-only inference is supported with degraded performance.
OE-2	The Professor and Student Web Portals shall operate on Google Chrome v110+, Mozilla Firefox v110+, and Microsoft Edge v110+ with JavaScript enabled.
OE-3	The Flutter mobile application shall support Android 10 (API level 29) or later and iOS 15 or later.

OE-4	The system shall function on the university intranet without requiring open internet access for core grading, submission, and proctoring features.
OE-5	Judge0 code compilation shall be deployed as a self-hosted Docker container. No third-party cloud execution APIs are used.
OE-6	All AI/ML processing including LLM inference, LIME/SHAP computation, and semantic embedding generation shall execute

3. Requirement Identifying Technique

The following techniques were applied to elicit, validate, and refine the requirements documented in this SRS:

3.1 Observations

A structured observation session was conducted during a live grading cycle at STMU. We observed one professor for an entire assignment grading period from receiving the first submission to publishing the final grade. Time, decision patterns, and inconsistencies introduced by collaboration were recorded.

3.2 Story Boarding

Story boards were produced for three highest-risk user workflows: (1) professor grading cycle from assignment creation through AI review to grade release, (2) student submission through XAI feedback viewing and dispute submission, and (3) proctored examination session from Focus Mode activation through violation detection to post-exam report. Story boards were reviewed with stakeholders to validate that proposed interface flows matched mental models.

4. Functional Requirements

This section specifies the functional requirements for each of the ten ProfessorOS modules. Each module includes: (4.x.1) System Functional Requirements consolidated table, (4.x.2) Use Case Modeling description, and (4.x.3) Use Case Description for the primary use case. A unified domain model, system activity diagram, and system sequence diagram are provided at the end of this section.

4.1 Functional Requirement 1: Authentication and User Management (M-01)

This module governs all user identity and access control functions. It establishes the security foundation on which every other module depends.

4.1.1 System Functional Requirements

FR-M01 – Authentication and User Management

Field	Description
Identifier	FR-M01
Title	Authentication and User Management
Requirement	The system shall provide secure user authentication and role-based access control for all platform users. The system shall support institutional email registration with email verification, secure session management, password recovery, biometric authentication on the mobile application, and role enforcement for Professors, Students, Teaching Assistants, and University Administrators. The system shall also support bulk student import through CSV upload functionality.
Source	Professor, Student, Teaching Assistant, University Administrator
Rationale	Secure identity verification and role enforcement are required to protect academic data, maintain grading integrity, and ensure authorised access to platform resources.
Business Rules	BR-1: Accounts shall be temporarily locked after multiple failed login attempts. BR-2: Only University Administrators may perform bulk student imports. BR-3: All authenticated sessions shall expire automatically after a predefined duration of inactivity.
Dependencies	None
Priority	High
Use Case Traceability	UC-1: Log In to ProfessorOS

FR-M02 – Course and Assignment Management

Field	Description
Identifier	FR-M02
Title	Course and Assignment Management
Requirement	The system shall enable professors to manage course assessments by creating, organizing, publishing, and archiving courses and their associated assignments, including the configuration of grading rubrics, assessment settings, submission requirements, deadlines, and grading policies.
Source	Professor
Rationale	A centralized course and assessment management module ensures that professors can efficiently organize course materials, maintain consistent evaluation standards, and manage assignments throughout their lifecycle.
Business Rules	BR-1: Every assignment shall have an associated grading rubric before publication. BR-2: Each rubric shall contain at least one evaluation criterion. BR-3: The total weight of all rubric criteria shall equal 100% before an assignment can be published.
Dependencies	FR-M01
Priority	High
Use Case Traceability	UC-2: Build Grading Rubric

FR-M03 – AI Grading and Evaluation

Field	Description
Identifier	FR-M03
Title	AI Grading and Evaluation

Field	Description
Requirement	The system shall support AI-assisted evaluation of student submissions by generating rubric-based grades, criterion-level feedback, and performance assessments, while enabling authorized academic staff to review, justify modifications to, approve, and release final grades before they become visible to students.
Source	Professor, Teaching Assistant
Rationale	AI-assisted grading improves evaluation efficiency and consistency while maintaining academic integrity through mandatory human review, approval, and oversight before grade publication.
Business Rules	BR-1: A justification shall be required for every modification to an AI-generated grade. BR-2: Only Professors may approve and release final grades. BR-3: Released grades may only be modified through the formal dispute resolution workflow.
Dependencies	FR-M01, FR-M02
Priority	High
Use Case Traceability	UC-3: Review and Override AI Grade

FR-M04 – Explainable AI (XAI) Processing

Field	Description
Identifier	FR-M04
Title	Explainable AI (XAI) Processing
Requirement	The system shall provide explainable AI feedback for AI-graded submissions. The system shall display visual explanations, criterion-level reasoning, and highlighted feedback indicators that help professors and students understand how grades were

Field	Description
	determined. Explanation results shall be retrievable for previously graded submissions.
Source	Professor, Student
Rationale	Transparent AI explanations improve trust, accountability, and understanding of automated grading decisions.
Business Rules	BR-1: Students may only access explainable AI feedback after grade approval. BR-2: Objective question types may display simplified explanation summaries instead of text highlights. BR-3: Explanation data shall be regenerated if previously stored explanation records expire or become unavailable.
Dependencies	FR-M03
Priority	High
Use Case Traceability	UC-4: View XAI Explanation as Student

FR-M05 – Student Submission Portal

Field	Description
Identifier	FR-M05
Title	Student Submission Portal
Requirement	Students shall access active assignments, submit coursework in multiple supported formats, receive grading feedback, view explainable AI results, and submit grade disputes within a configurable dispute period. The system shall also provide notifications related to assignments, deadlines, grades, and dispute responses.
Source	Student

Field	Description
Rationale	A unified submission portal improves accessibility, submission management, and communication between students and instructors.
Business Rules	BR-1: Late submissions shall only be accepted within the configured late submission policy. BR-2: Submission timestamps shall be recorded automatically by the system. BR-3: File uploads shall be restricted by supported file types and maximum size limits. BR-4: Grade disputes must be submitted within the configured dispute window.
Dependencies	FR-M01, FR-M02, FR-M03, FR-M04
Priority	High
Use Case Traceability	UC-5: Submit Assignment

FR-M06 – Analytics Dashboard

Field	Description
Identifier	FR-M06
Title	Analytics Dashboard
Requirement	The system shall provide professors with real-time academic performance analytics including grade distributions, rubric performance summaries, at-risk student identification, and cohort performance trends through interactive dashboard visualisations.
Source	Professor
Rationale	Academic analytics support early intervention, improve course monitoring, and help professors identify student performance issues more effectively.

Field	Description
Business Rules	BR-1: At-risk thresholds shall be configurable per course.BR-2: At-risk status shall update automatically after grade approval.BR-3: Analytics access shall be restricted to authorised course personnel.
Dependencies	FR-M01, FR-M02, FR-M03
Priority	High
Use Case Traceability	UC-6: Identify At-Risk Students

FR-M07 – Smart Proctoring

Field	Description
Identifier	FR-M07
Title	Smart Proctoring
Requirement	The system shall provide lightweight examination monitoring capabilities through the mobile application by detecting focus violations, recording examination session events, and notifying professors of suspicious examination behaviour during active assessments.
Source	Student, Professor
Rationale	Non-invasive monitoring mechanisms help discourage cheating behaviour while maintaining student privacy.
Business Rules	BR-1: Multiple unresolved focus violations shall automatically flag an examination session for review.BR-2: Recorded violation logs shall be immutable after creation.BR-3: Offline violations shall synchronise automatically after network reconnection.
Dependencies	FR-M01, FR-M02
Priority	High

Field	Description
Use Case Traceability	UC-7: Detect and Log Focus Violation

FR-M08 – Code Editor and Compiler

Field	Description
Identifier	FR-M08
Title	Code Editor and Compiler
Requirement	The system shall provide an integrated browser-based programming environment supporting C, C++, Python, Java, JavaScript, Go, Rust, PHP, Ruby, C#, TypeScript, and Swift. The environment shall support code execution, compilation feedback, test case evaluation, and assignment submission for programming-based coursework. Professors shall be able to define hidden test cases for automated evaluation.
Source	Student, Professor
Rationale	Integrated programming environments simplify code submission workflows and support objective evaluation of programming assignments.
Business Rules	BR-1: Executing code shall not automatically create a submission.BR-2: Students may execute code multiple times before final submission.BR-3: Hidden test cases shall remain inaccessible to students.
Dependencies	FR-M01, FR-M02, FR-M05
Priority	High
Use Case Traceability	UC-8: Execute Code and View Results

FR-M09 – AI Teaching Assistant

Field	Description
Identifier	FR-M9
Title	AI Teaching Assistant

Field	Description
Requirement	The system shall provide a course-specific conversational AI Teaching Assistant capable of answering student questions using instructor-provided learning materials. Responses shall include references to relevant course resources and shall exclude restricted academic content such as examination answers, assignment solutions, and grade dispute discussions.
Source	Student, Professor
Rationale	A course-aware AI assistant improves student support availability while ensuring responses remain grounded in instructor-approved academic materials.
Business Rules	BR-1: The AI Teaching Assistant shall not answer restricted academic integrity-related questions.BR-2: Responses shall include references to relevant source materials.BR-3: Course knowledge repositories shall update when new materials are uploaded.
Dependencies	FR-M01, FR-M02
Priority	Medium
Use Case Traceability	UC-9: Ask AI Teaching Assistant a Course Question

4.2 Use Case Modeling

The following diagrams present the individual use case models for each of the ten ProfessorOS modules. Each diagram shows the actors and use cases specific to that module, with include and extend relationships where applicable.

4.3 Use Case Descriptions and Diagrams

This section presents the detailed use case specifications for the primary interactions within the ProfessorOS platform. Each use case defines the participating actors, triggering conditions,

preconditions, postconditions, normal operational flow, alternative flows, exception handling, and associated business rules.

UC-1: Log In to ProfessorOS

Field	Description
Use Case ID	UC-1
Use Case Name	Log In to ProfessorOS
Actors	Primary: Professor, Student, Teaching Assistant, University Administrator
Description	A registered user authenticates into the system using institutional credentials and is redirected to the appropriate dashboard according to their assigned role.
Trigger	User submits login credentials on the login page.
Preconditions	PRE-1: User account exists and is verified. PRE-2: User account is active and not suspended.
Postconditions	POST-1: User session is successfully created. POST-2: User is redirected to the role-specific dashboard. POST-3: Login activity is recorded in the audit log.
Normal Flow	1. User enters institutional email address and password. 2. System validates the submitted credentials. 3. System creates an authenticated session. 4. System identifies the user role. 5. User is redirected to the appropriate dashboard.
Alternative Flows	1.A: Mobile biometric authentication may be used instead of password entry. Upon successful verification, the system authenticates the user and continues from Step 3.
Exceptions	E1: Invalid credentials entered. E2: Email account is not verified. E3: User account is suspended. E4: Maximum login attempts exceeded.

Field	Description
Business Rules	BR-1: User accounts shall be temporarily locked after multiple failed login attempts. BR-2: Authenticated sessions shall expire automatically after a configured duration. BR-3: Only verified institutional accounts may access the system.

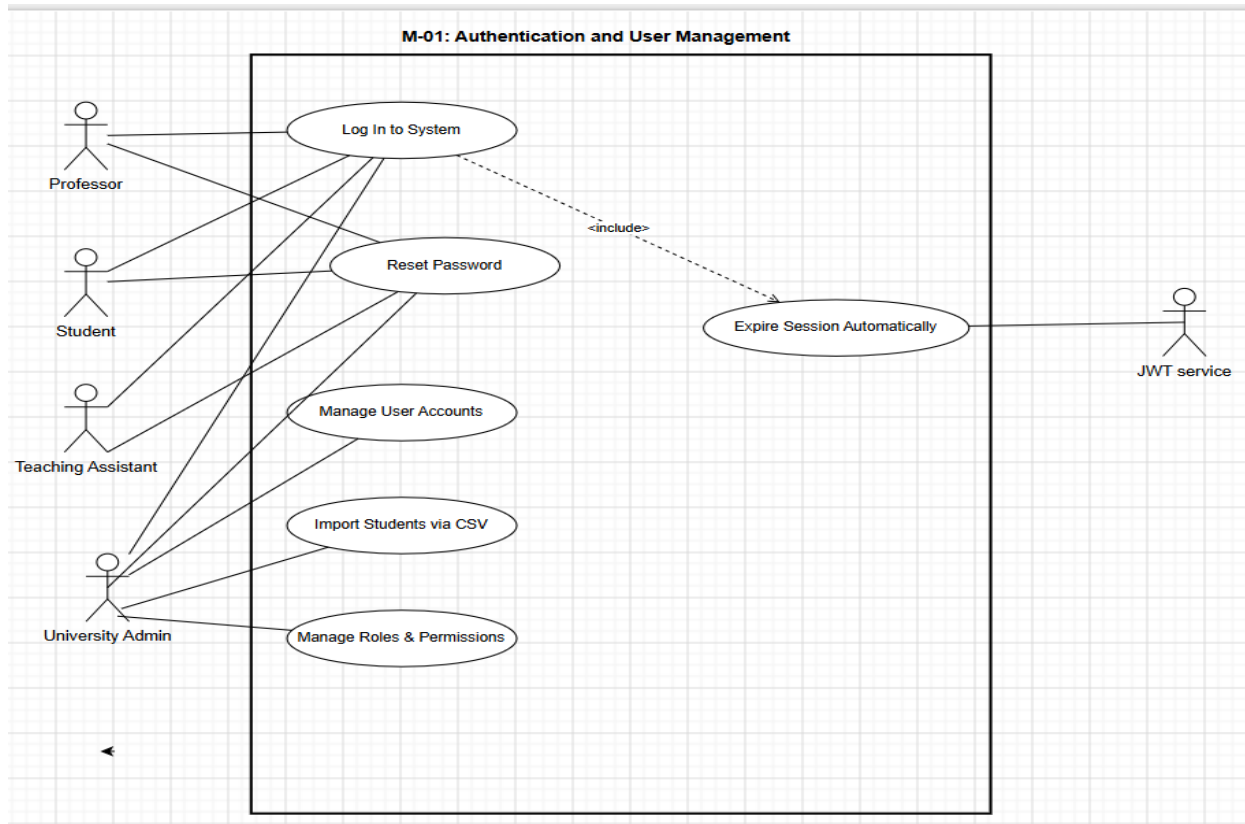


Figure 4.3.1: M-01 Use Case Diagram

UC-2: Build Grading Rubric for an Assignment

Field	Description
Use Case ID	UC-2
Use Case Name	Build Grading Rubric for an Assignment
Actors	Primary: Professor

Field	Description
Description	A professor creates a structured grading rubric by defining criteria, performance levels, and weight distributions for an assignment.
Trigger	Professor selects the rubric creation option for an assignment.
Preconditions	PRE-1: Assignment exists in draft state. PRE-2: Professor is assigned to the course.
Postconditions	POST-1: Rubric is stored and linked to the assignment. POST-2: Assignment becomes eligible for publication once rubric validation succeeds.
Normal Flow	1. Professor opens the rubric builder. 2. Professor adds grading criteria. 3. Professor defines performance level descriptions for each criterion. 4. Professor assigns weight percentages to criteria. 5. System validates total weight distribution. 6. Professor saves the rubric. 7. System associates the rubric with the assignment.
Alternative Flows	2.A: Professor clones an existing rubric from a previous assignment and modifies it as required before saving.
Exceptions	E1: Total criterion weights exceed or fall below the required value. E2: Required rubric fields are incomplete.
Business Rules	BR-1: Each rubric must contain at least one criterion. BR-2: Rubric weights must total exactly 100% before publication. BR-3: Rubrics may contain a maximum of ten criteria.

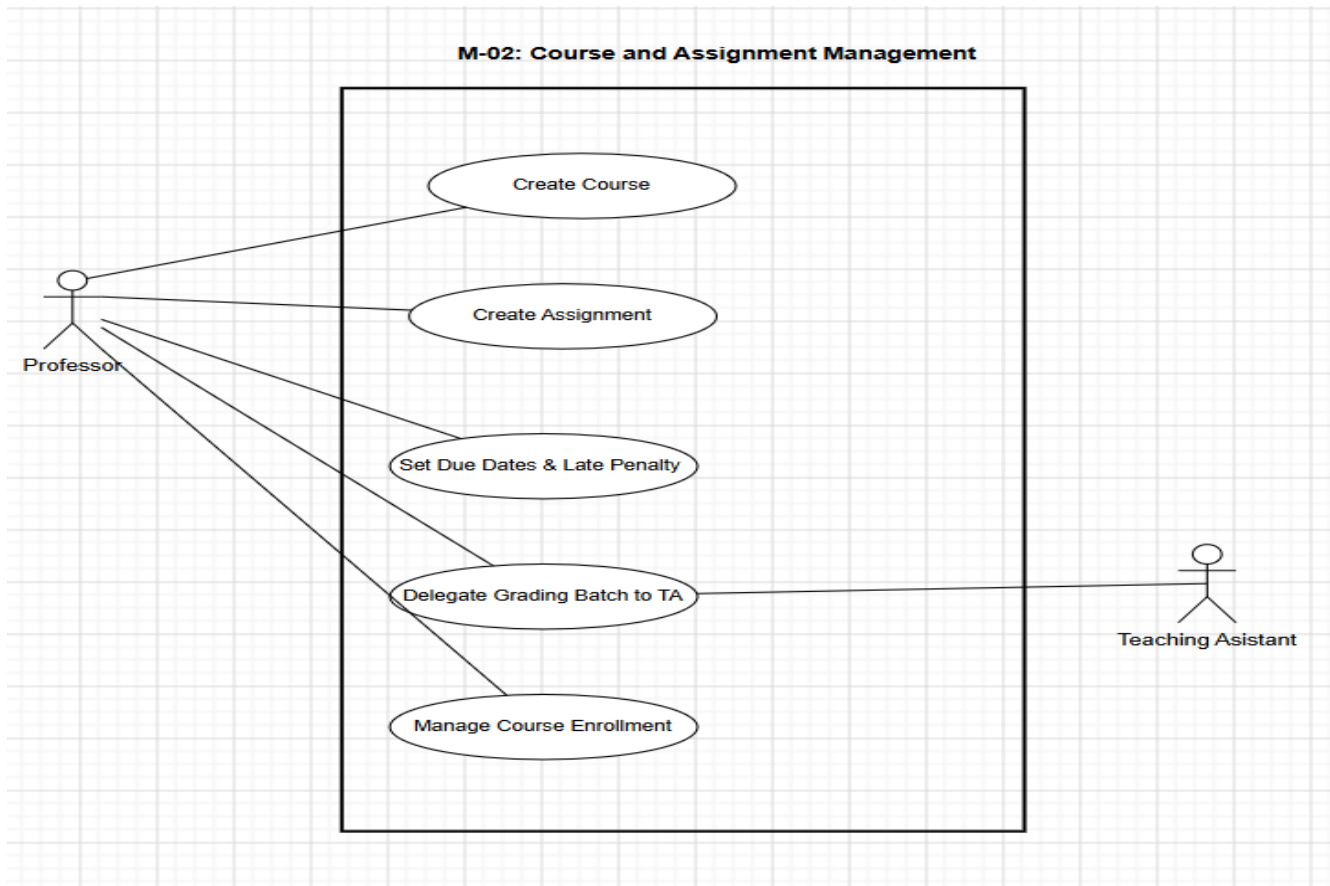


Figure 4.3.2: M-02 Use Case Diagram

UC-3: Review and Override AI Grade

Field	Description
Use Case ID	UC-3
Use Case Name	Review and Override AI Grade
Actors	Primary: Professor Secondary: Teaching Assistant
Description	A professor or delegated Teaching Assistant reviews AI-generated grades, feedback, and explanations before approving or modifying the final evaluation.

Field	Description
Trigger	AI grading process completes for a student submission.
Preconditions	PRE-1: AI grading results are available. PRE-2: User is authorised to access the course grading panel.
Postconditions	POST-1: Submission status is updated. POST-2: Approved grades become visible to students. POST-3: Override actions are recorded in the audit log.
Normal Flow	1. User opens the grading review panel. 2. System displays submission details, scores, and AI feedback. 3. User reviews criterion-level evaluations. 4. User approves the generated grade. 5. System releases the approved result to the student.
Alternative Flows	4.A: User overrides one or more criterion scores by entering revised values and written justification before approval.
Exceptions	E1: Override value exceeds assignment grading limits. E2: Submission is flagged for additional review instead of approval.
Business Rules	BR-1: Written justification is mandatory for all overrides. BR-2: Only Professors may release final grades. BR-3: Released grades may only be modified through the dispute process.

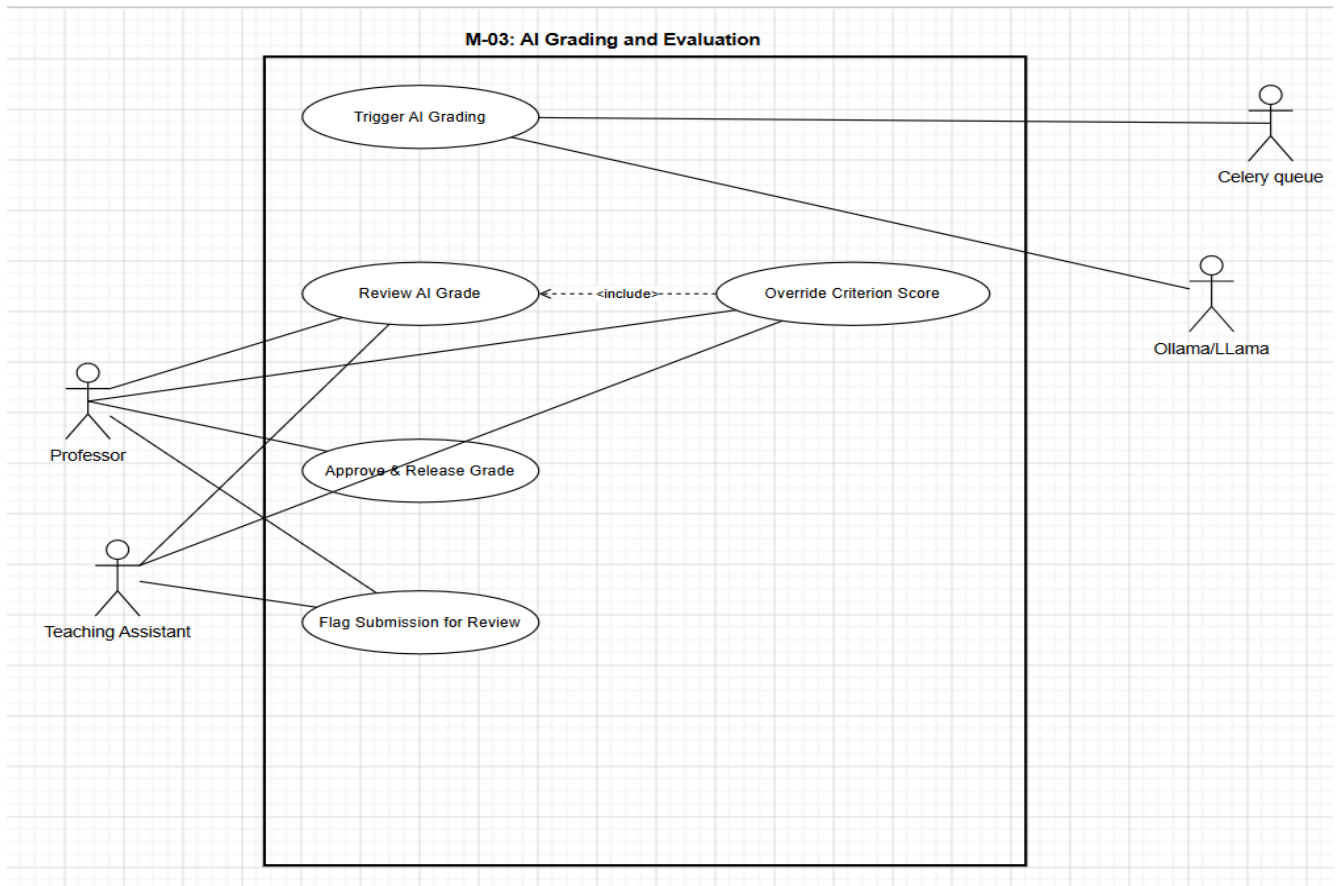


Figure 4.3.3: M-03 Use Case Diagram

UC-4: View XAI Explanation as Student

Field	Description
Use Case ID	UC-4
Use Case Name	View XAI Explanation as Student
Actors	Primary: Student
Description	A student views detailed grading feedback and explainable AI visualisations for an approved submission.
Trigger	Student selects the feedback option for a graded submission.

Field	Description
Preconditions	PRE-1: Submission grade has been approved. PRE-2: Student owns the submission being viewed.
Postconditions	POST-1: Student reviews criterion-level feedback and explanations. POST-2: Feedback access activity is logged.
Normal Flow	1. Student opens the feedback viewer. 2. System retrieves grading feedback and explanation data. 3. System displays submission highlights and criterion-level explanations. 4. Student reviews scores, feedback, and visual indicators. 5. System provides access to the dispute submission option.
Alternative Flows	2.A: If explanation data is unavailable, the system generates the explanation before displaying the results.
Exceptions	E1: Objective assessment types display simplified feedback summaries instead of textual explanations.
Business Rules	BR-1: Students may only access feedback for their own submissions. BR-2: Feedback is available only after grade approval. BR-3: Explanation records may be regenerated if unavailable or expired.

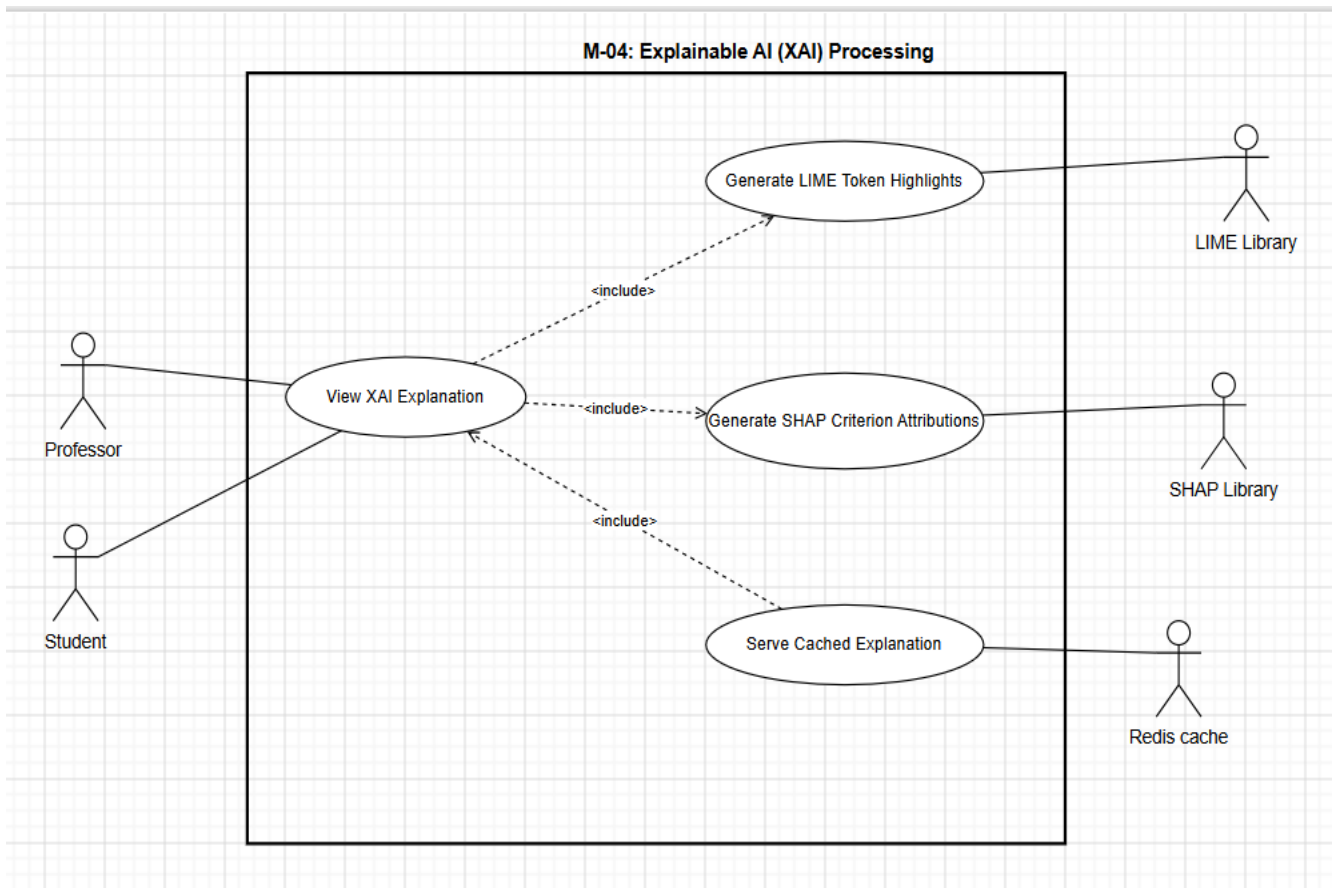


Figure 4.3.4: M-04 Use Case Diagram

UC-5: Submit Assignment

Field	Description
Use Case ID	UC-5
Use Case Name	Submit Assignment
Actors	Primary: Student
Description	A student submits coursework through the platform using the supported submission format for the assignment.
Trigger	Student clicks the submission option for an active assignment.

Field	Description
Preconditions	PRE-1: Assignment is published and accessible. PRE-2: Student is enrolled in the course. PRE-3: Submission attempts are permitted.
Postconditions	POST-1: Submission is stored successfully. POST-2: Submission timestamp is recorded. POST-3: Submission is forwarded for grading. POST-4: Confirmation message is displayed.
Normal Flow	1. Student opens the assignment page. 2. System displays the appropriate submission interface. 3. Student enters or uploads assignment content. 4. Student submits the assignment. 5. System validates submission requirements. 6. System records the submission and timestamp. 7. System applies any applicable late penalty. 8. System forwards the submission for grading. 9. System confirms successful submission.
Alternative Flows	3.A: For programming assignments, the student may execute code multiple times before submitting the final version.
Exceptions	E1: Uploaded file exceeds permitted size limits. E2: Submission content is empty. E3: Assignment is unavailable or unpublished.
Business Rules	BR-1: Late submissions are accepted only within the configured late policy. BR-2: Submission timestamps are generated by the system. BR-3: Only supported file formats may be uploaded.

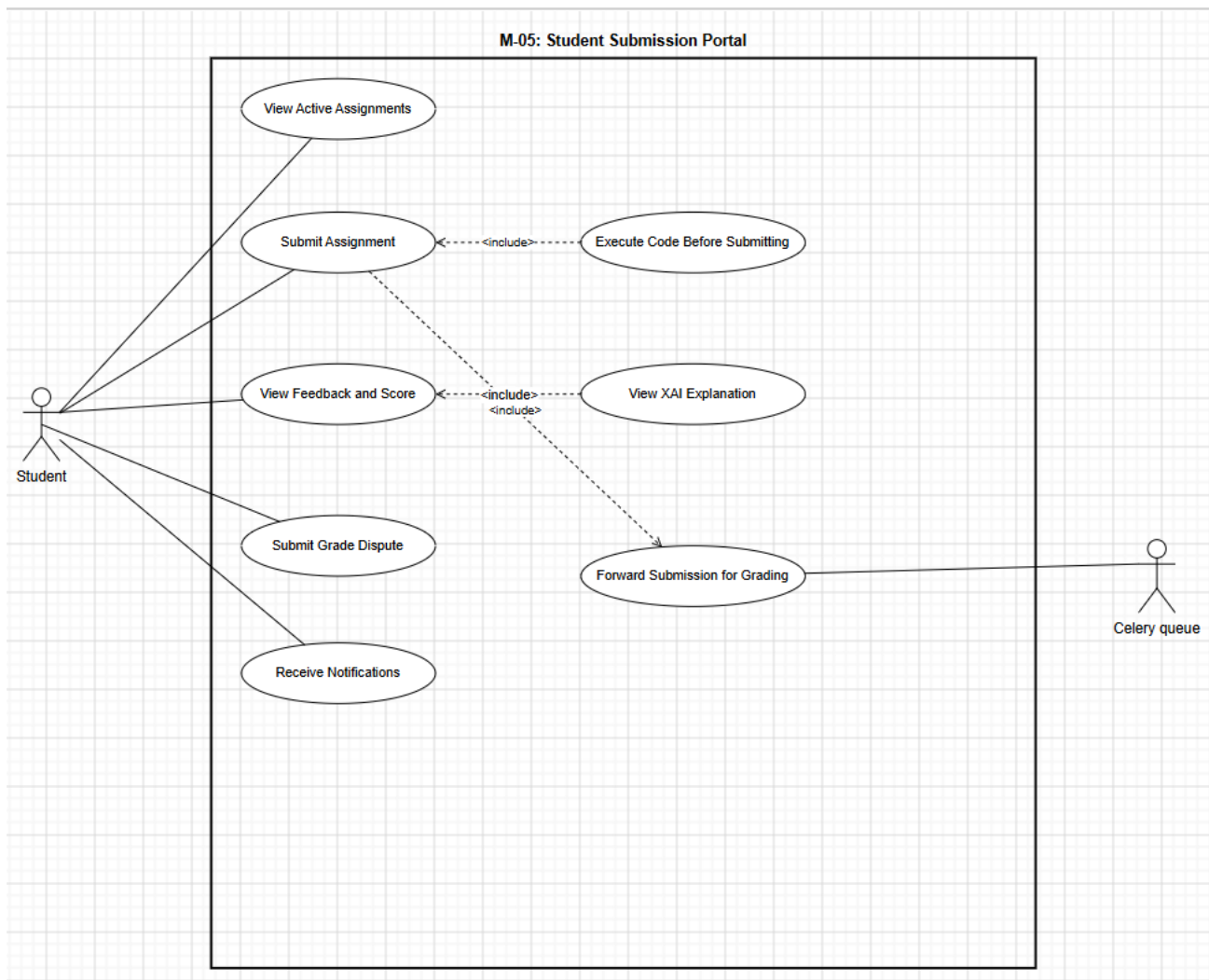


Figure 4.3.5: M-05 Use Case Diagram

UC-6: Identify At-Risk Students

Field	Description
Use Case ID	UC-6
Use Case Name	Identify At-Risk Students
Actors	Primary: Professor

Field	Description
Description	A professor reviews course analytics to identify students requiring academic intervention.
Trigger	Professor opens the at-risk student analytics view.
Preconditions	PRE-1: Course contains approved grading data. PRE-2: Professor is assigned to the course.
Postconditions	POST-1: Professor reviews identified students and related performance information. POST-2: Professor may initiate communication with flagged students.
Normal Flow	1. System displays the at-risk student dashboard. 2. Professor reviews student performance indicators. 3. Professor opens detailed student performance history. 4. Professor optionally sends a notification or message to the student.
Alternative Flows	None.
Exceptions	E1: No students currently meet the at-risk criteria.
Business Rules	BR-1: Risk thresholds are configurable per course. BR-2: Risk evaluation updates automatically after grade approval. BR-3: Analytics access is restricted to authorised personnel.

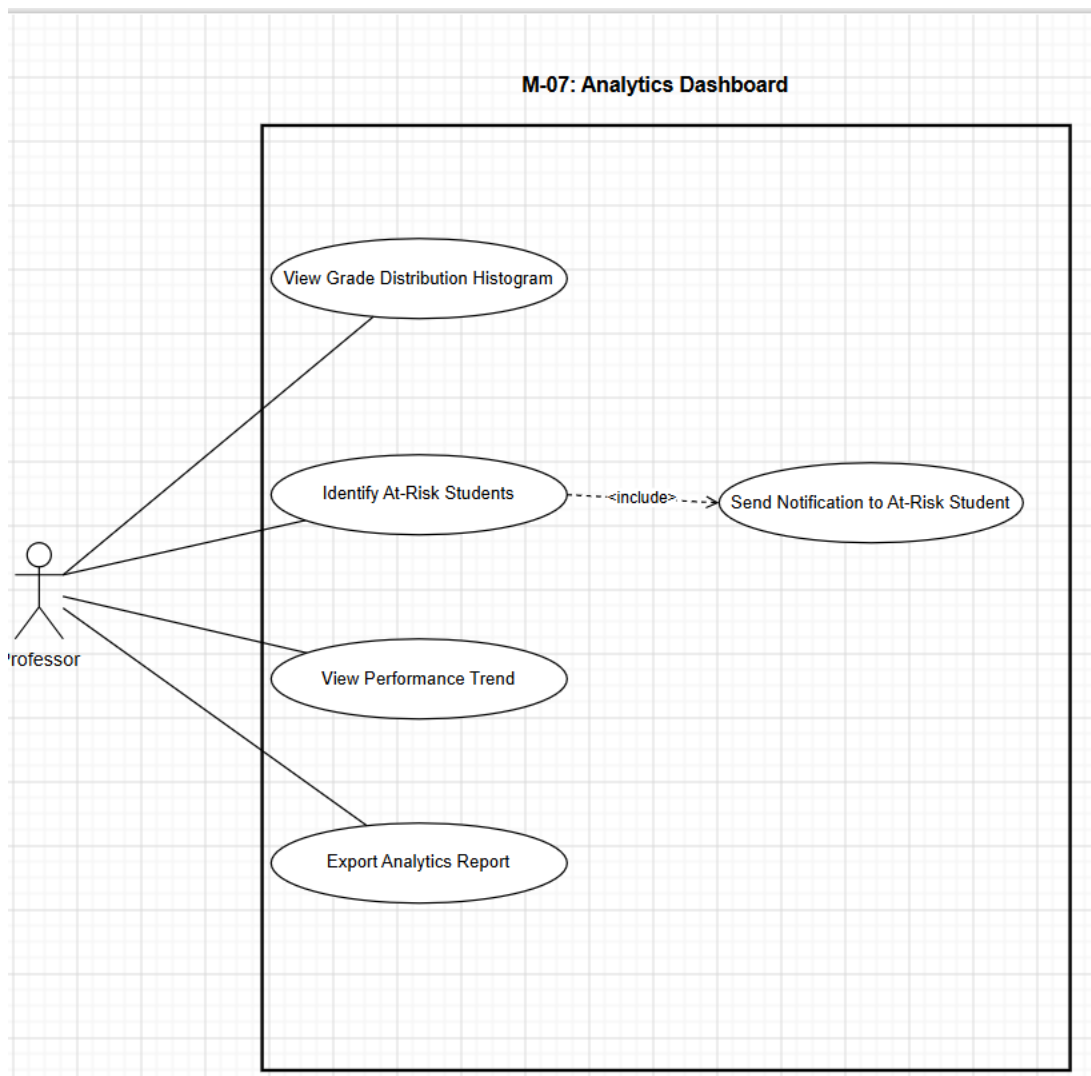


Figure 4.3.6: M-06 Use Case Diagram

UC-7: Detect and Log Focus Violation

Field	Description
Use Case ID	UC-7
Use Case Name	Detect and Log Focus Violation
Actors	Primary: Student

Field	Description
Description	The system detects examination focus violations during a proctored assessment session and records the incident for professor review.
Trigger	Student exits or switches away from the active examination session.
Preconditions	PRE-1: A proctored examination session is active. PRE-2: Student is participating in Focus Mode.
Postconditions	POST-1: Violation event is recorded. POST-2: Professor dashboard is updated. POST-3: Student receives a violation warning.
Normal Flow	1. System detects a focus violation event. 2. System records the violation locally and on the server. 3. System updates the violation counter. 4. System notifies the professor dashboard. 5. System displays a return warning overlay to the student.
Alternative Flows	2.A: If the device is offline, the violation is stored locally and synchronised after reconnection.
Exceptions	E1: Network connection is unavailable during synchronisation.
Business Rules	BR-1: Multiple unresolved violations automatically flag the session for review. BR-2: Violation records cannot be deleted or modified. BR-3: Offline violations must synchronise in chronological order.

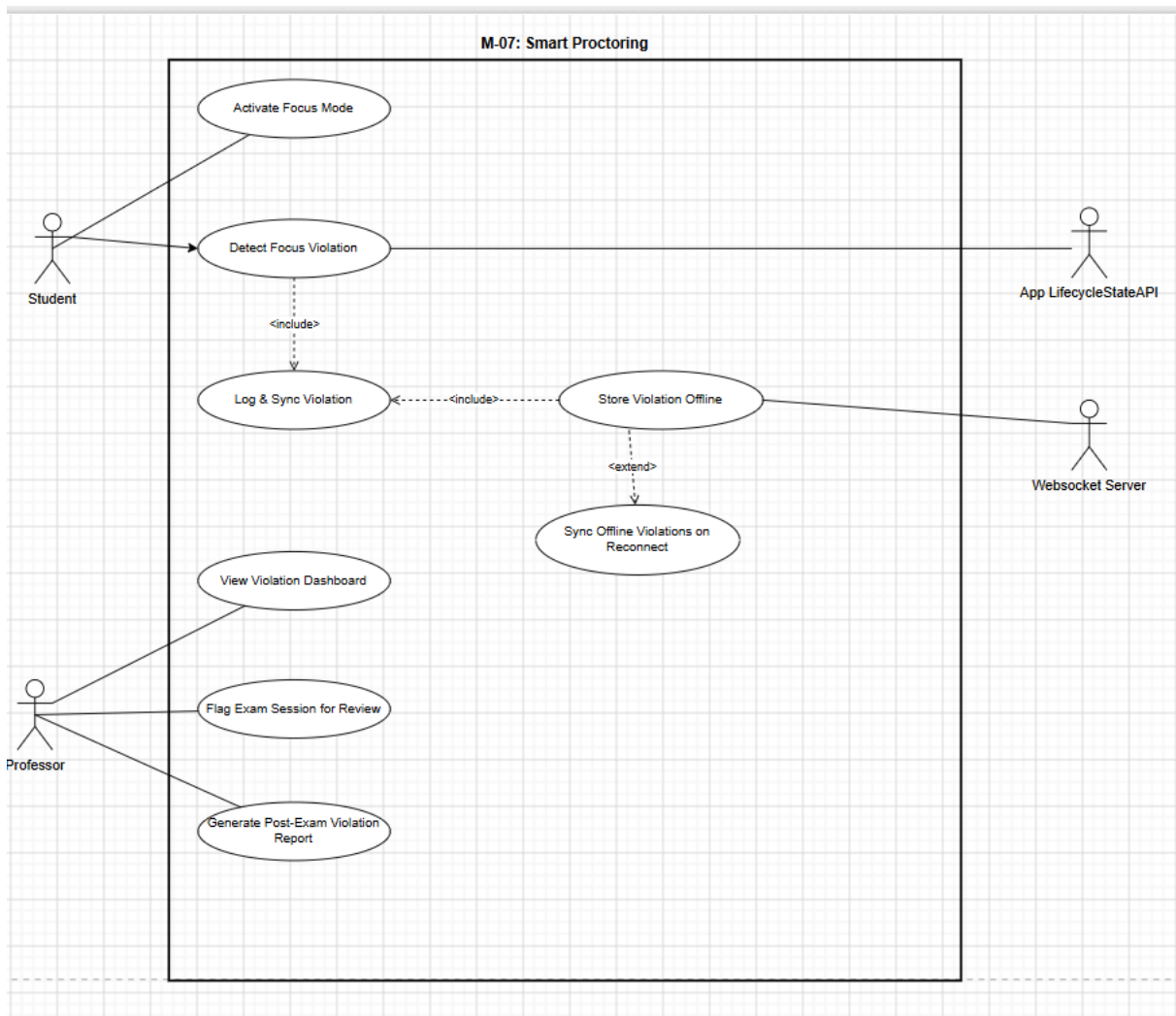


Figure 4.3.7: M-07 Use Case Diagram

UC-8: Execute Code and View Results

Field	Description
Use Case ID	UC-8
Use Case Name	Execute Code and View Results
Actors	Primary: Student

Field	Description
Description	A student executes code within the integrated programming environment and reviews compilation and execution results.
Trigger	Student clicks the Run button in the programming editor.
Preconditions	PRE-1: Assignment type is programming-based. PRE-2: Student has access to the assignment.
Postconditions	POST-1: Execution output is displayed. POST-2: Test case results are updated.
Normal Flow	1. Student writes or pastes source code into the editor. 2. Student selects a programming language. 3. Student executes the code. 4. System processes the execution request. 5. System retrieves execution results. 6. System displays compilation output, execution statistics, and test case outcomes.
Alternative Flows	None.
Exceptions	E1: Compilation failure occurs. E2: Program execution exceeds the allowed limit. E3: Code execution service is unavailable.
Business Rules	BR-1: Executing code does not create a final submission. BR-2: Students may test code multiple times before submission. BR-3: Hidden evaluation test cases remain inaccessible to students.

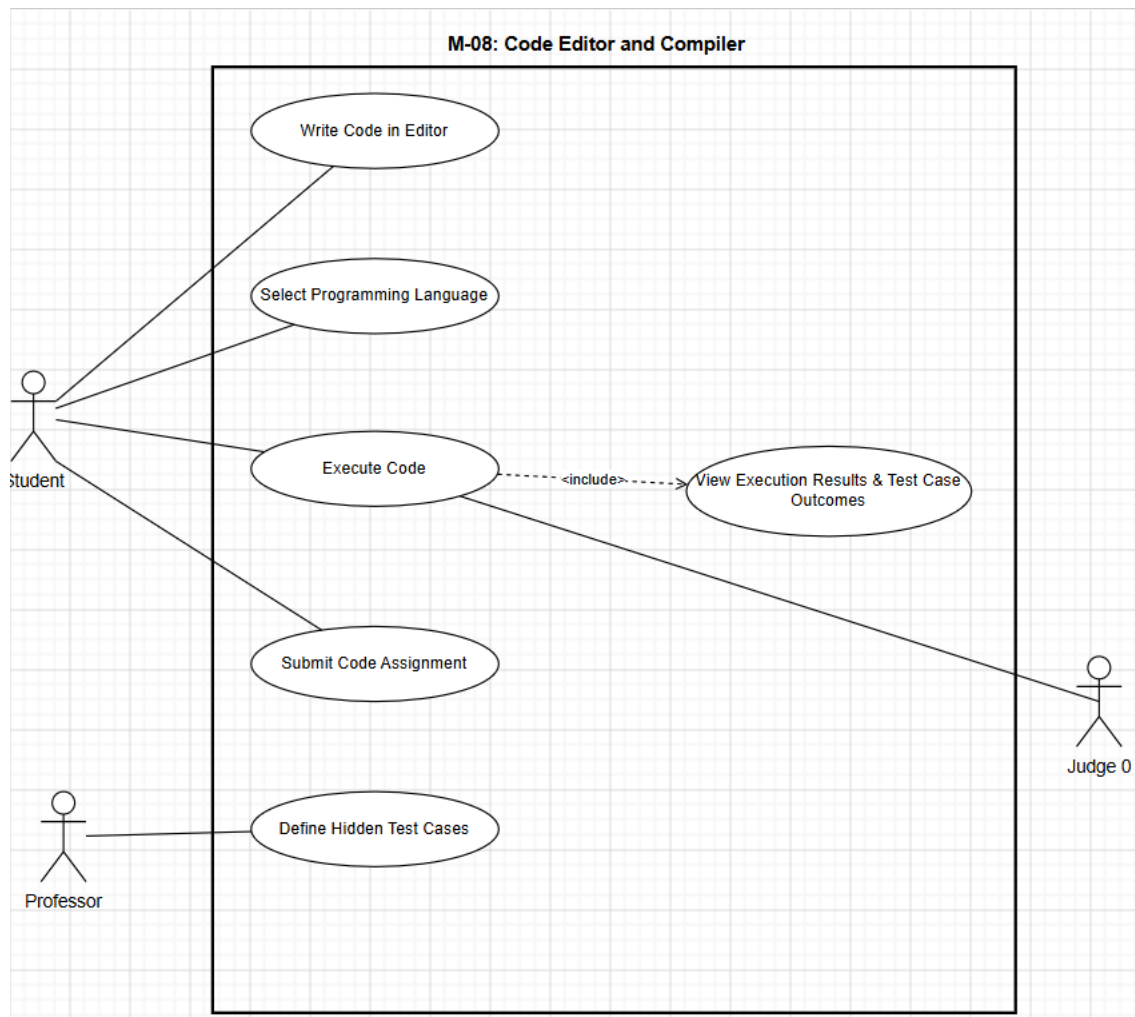


Figure 4.3.8: M-08 Use Case Diagram

UC-9: Ask AI Teaching Assistant a Course Question

Field	Description
Use Case ID	UC-9
Use Case Name	Ask AI Teaching Assistant a Course Question
Actors	Primary: Student

Field	Description
Description	A student submits a course-related question to the AI Teaching Assistant and receives a response generated from instructor-provided learning materials.
Trigger	Student submits a question in the AI Teaching Assistant interface.
Preconditions	PRE-1: Course learning materials have been uploaded. PRE-2: Student is enrolled in the course.
Postconditions	POST-1: Student receives a contextual response with supporting references. POST-2: Interaction is logged for monitoring purposes.
Normal Flow	1. Student enters a question in the AI Teaching Assistant interface. 2. System retrieves relevant course material. 3. System generates a contextual response. 4. System displays the response with source references.
Alternative Flows	None.
Exceptions	E1: No relevant course material is available for the query. E2: AI response service is temporarily unavailable.
Business Rules	BR-1: The AI Teaching Assistant shall not provide examination answers, assignment solutions, or grade dispute decisions. BR-2: Responses shall include references to course materials. BR-3: Conversation context may be maintained for multi-turn interactions.

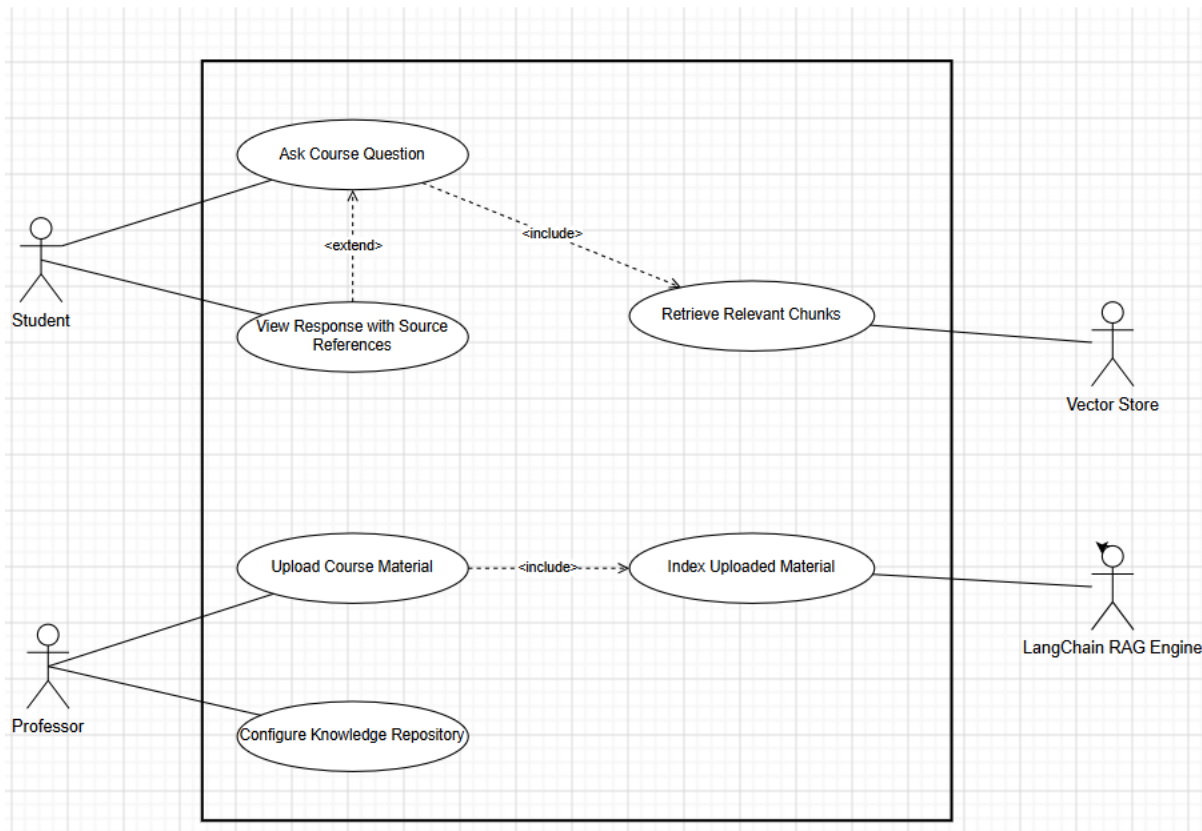


Figure 4.3.9: M-9 Use Case Diagram

4.4 Domain Model

The domain model represents the principal concepts and their relationships in the ProfessorOS system. It provides a high-level conceptual understanding of how domain entities interact to support the system's functionality, independent of technical or implementation details. The User is the central entity, specialized into four subtypes Professor, Student, Teaching Assistant, and University Administrator each inheriting common authentication attributes and differing in role-based access.

A Course is created and owned by a Professor and contains one or more Assignments. Each Assignment has exactly one Rubric, which is composed of one to ten Rubric Criteria. When a Student submits work against an Assignment, a Submission record is created and triggers one AIGradeResult (via the AI

grading pipeline) and one XAIExplanation (via LIME and SHAP post-processing). Each Rubric Criterion within the AIGradeResult carries an individual criterion score, a performance level label, and AI-generated feedback text.

During proctored examinations, a Proctoring Session is created per student per exam and accumulates ViolationEvent records triggered by app-switch detection. For code-type assignments, a Code Execution Result is produced by Judge0 for each Run action and linked to the Submission on final submission. Finally, each Course maintains a collection of Course Material documents that are indexed into the FAISS vector store and consulted by the AI Teaching Assistant during Session interactions.

Figure: Unified System Domain Model — ProfessorOS

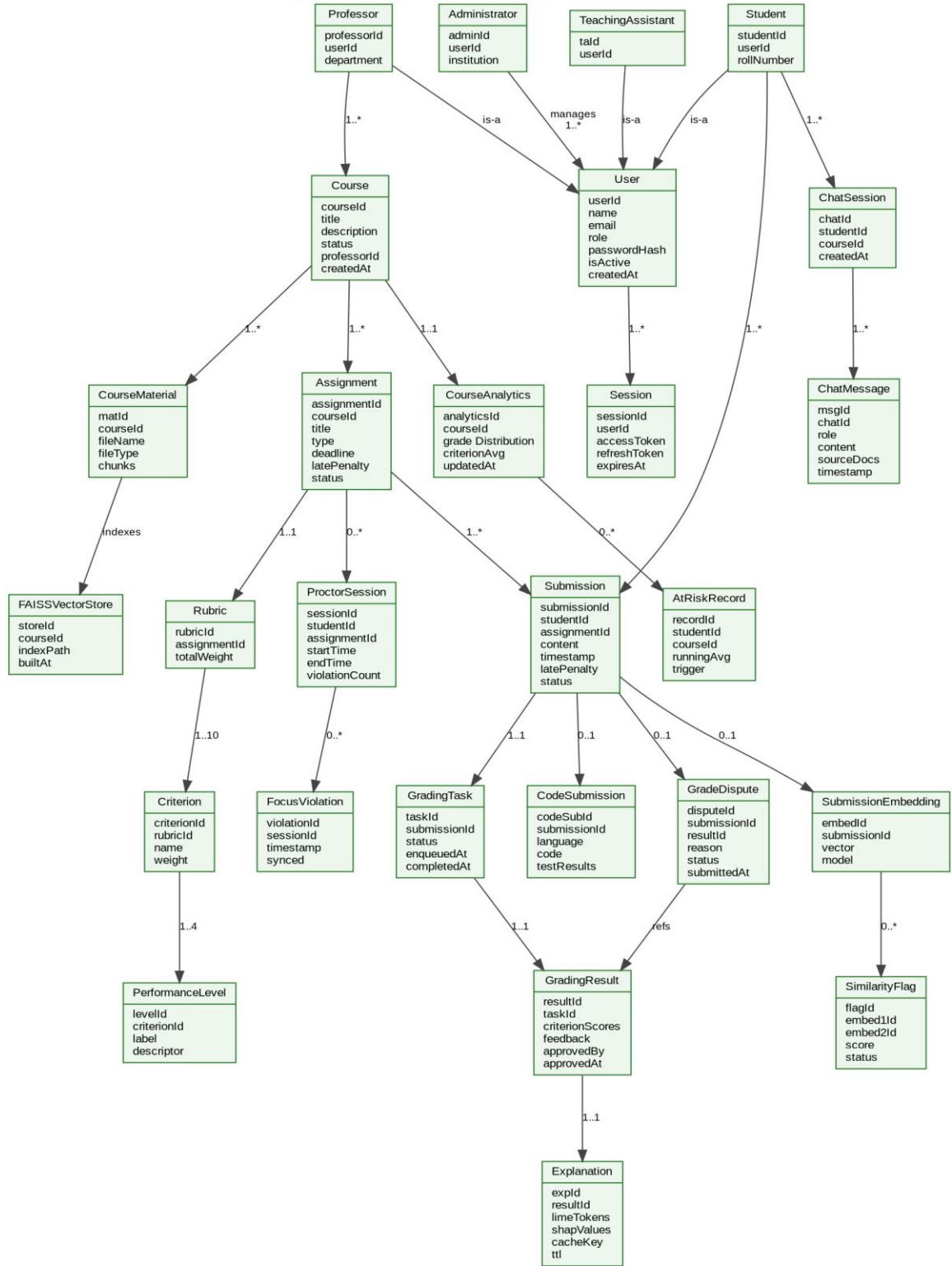


Figure 4.4.1: ProfessorOS Domain Model

5. Non-Functional Requirements

5.1 Selection of Quality Attributes

The following quality attributes were selected because they directly support the objectives of ProfessorOS and ensure that the platform meets the operational needs of universities.

Quality Attribute	Why it was Selected
Reliability	The platform manages assignments, examinations, and grades. Reliable operation is essential to prevent data loss and ensure uninterrupted academic activities.
Performance	Professors and students expect quick responses when uploading assignments, viewing grades, or accessing dashboards, especially during peak academic periods.
Security	ProfessorOS stores sensitive academic records and personal information. Appropriate security controls are required to prevent unauthorized access and protect user data.
Usability	The platform will be used by individuals with different levels of technical expertise. A simple and intuitive interface reduces the learning curve and improves productivity.
Maintainability	The system is expected to evolve after deployment. A maintainable design simplifies debugging, feature additions, and future enhancements.
Scalability	The number of users and courses may increase over time. The system should continue to perform efficiently as demand grows.
Availability	Students and instructors require access to the platform during assignment deadlines and examinations. High availability minimizes disruptions to academic activities.
Compatibility	Users access the platform from different operating systems, browsers, and devices. The system should provide a consistent experience across supported environments.

5.2 Reliability (NFR-R1)

Requirement

The system shall maintain 99.5% uptime during scheduled operational hours (6:00 AM to 11:00 PM local time) and shall recover from any unplanned failure within 30 minutes, with no loss of submitted assessment data.

Rationale

ProfessorOS manages time-sensitive academic workflows including assignment submissions, live examinations, and grade releases. An unreliable system can directly harm students' academic outcomes if submissions are lost or exams cannot be completed. Consistent uptime builds institutional trust and ensures the platform can be adopted as a mission-critical tool.

5.3Performance (NFR-P1)

Requirement

The system shall deliver AI-generated criterion-level grading feedback to students within 60 seconds of submission and shall serve all standard page requests within 3 seconds, under a concurrent load of at least 180 active users, without degradation in output quality or accuracy.

Rationale

The primary business objective of ProfessorOS is to reduce the 12–18 hour per-week grading burden observed at STMU by at least 75%. This reduction is only realised if AI feedback arrives faster than manual marking cycles. The 60-second threshold is derived from BO-2 and represents the maximum tolerable wait for both students and professors before the workflow is abandoned. The 180-user figure matches the maximum STMU cohort size per assignment cycle. Satisfying this requirement directly validates the core value proposition of the platform during the highest-demand periods, such as end-of-semester submission deadlines, without requiring additional server infrastructure beyond the single Docker Compose deployment.

5.4Security (NFR-S1)

Requirement

The system shall enforce role-based access control (RBAC) ensuring that each user can access only the resources and operations assigned to their role, shall store all passwords using the Argon2id hashing algorithm, shall transmit all data over HTTPS with TLS 1.3, and shall sanitise all user inputs to prevent injection attacks across every module.

Rationale

ProfessorOS processes sensitive academic records including grades, rubrics, dispute submissions, and student personal information. A breach or unauthorised grade modification would constitute an academic integrity violation with serious institutional consequences. RBAC ensures that students cannot access grading panels, teaching assistants cannot release final grades, and administrators cannot view individual submission content outside their scope. Argon2id is the current password-hashing recommendation of

OWASP and is resistant to GPU-based brute-force attacks. TLS 1.3 eliminates legacy cipher vulnerabilities present in earlier protocol versions. Input sanitisation prevents SQL injection and prompt-injection attacks against the LLM grading pipeline, which is a realistic threat surface for an AI-integrated academic system.

5.5 Usability (NFR-U1)

Requirement

At least 90 percent of first-time users across all four role categories shall be able to independently complete their primary task—professor creating and publishing an assignment, student submitting work and viewing XAI feedback, teaching assistant reviewing an AI grade, and administrator importing students via CSV—without external assistance after a single 10-minute orientation session.

Rationale

ProfessorOS will be used by individuals with widely varying technical backgrounds, from non-technical university administrators to students with limited software experience. The observation at STMU confirmed that professors are willing to adopt new tools only if the learning curve does not add to their existing workload. A system that is difficult to use will be abandoned regardless of its AI capability, nullifying all performance and accuracy gains. The 90 percent threshold and 10-minute orientation bound are derived from BO-7 and ensure the platform can be rolled out across departments without dedicated IT training programmes.

5.6 Maintainability (NFR-M1)

Requirement

The system shall be designed using a modular architecture such that any individual module can be updated, replaced, or disabled without affecting the operation of other modules. All modules shall include structured logging and expose health-check endpoints consumable by standard monitoring tools.

Rationale

ProfessorOS is expected to evolve continuously after initial deployment, with new AI models, regulatory requirements, and feature requests emerging over time. A maintainable architecture reduces the cost and risk of changes, shortens the bug-fix cycle, and allows the development team to add or update capabilities without system-wide regression.

5.7 Scalability (NFR-SC1)

Requirement

The system shall support horizontal scaling of all stateless services and shall handle a 300% surge in concurrent users above the baseline load without degradation in response time beyond 20% of the baseline median.

Rationale

Academic usage patterns are highly uneven, with traffic spikes occurring during exam periods, assignment deadlines, and grade-release windows. The ability to scale out dynamically ensures that performance SLAs are met during peak load without over-provisioning infrastructure during quiet periods, keeping operational costs proportional to actual demand.

5.8 Availability (NFR-A1)

Requirement

Critical user-facing services (submission portal, examination engine, and grading dashboard) shall remain available during rolling deployments and planned maintenance windows through zero-downtime deployment strategies. Scheduled maintenance shall be communicated to users at least 48 hours in advance.

Rationale

Students and instructors require uninterrupted access to the platform, particularly around assessment windows that cannot be rescheduled. Zero-downtime deployments and advance maintenance notices prevent unexpected disruptions, protect student outcomes, and maintain institutional confidence in the platform as a dependable academic infrastructure tool.

5.9 Compatibility (NFR-C1)

Requirement

The web application shall function correctly on the two most recent major versions of Chrome, Firefox, Safari, and Edge on both Windows and macOS. The Flutter mobile application shall support Android 10 (API level 29) and above, and iOS 15 and above.

Rationale

ProfessorOS serves a diverse user base spanning students, professors, teaching assistants, and administrators who access the platform from a wide variety of personal and institutional devices. Broad browser and OS compatibility ensures equitable access regardless of hardware or software choices, and reduces support burden from compatibility-related issues.

6. External Interface Requirements

6.1 User Interfaces Requirements

The following standards and constraints apply to all user interface components across the Professor Portal, Student Portal, and Flutter Mobile Application:

- UI-1: All web interfaces shall use Tailwind CSS 3.4 utility classes for consistent spacing, typography, and colour application. No custom CSS overrides shall be applied to Tailwind base classes.
- UI-2: All interactive elements shall meet WCAG 2.1 Level AA contrast ratio requirements (minimum 4.5:1 for normal text, 3:1 for large text).
- UI-3: The grading review panel shall use a split-panel layout: submission text with SHAP highlights on the left (60% width), criterion cards on the right (40% width) on screen widths of 1280 pixels or wider. Below 1280 pixels, panels shall stack vertically.
- UI-4: All forms shall display inline validation error messages adjacent to the field in error without requiring page reload. Fields shall re-validate on blur.
- UI-5: A loading skeleton or spinner shall be displayed for all operations expected to take longer than 500 milliseconds.
- UI-6: The Flutter mobile application bottom navigation bar shall contain exactly four items: Courses, Assignments, AI Assistant, and Profile.
- UI-7: Screen mock-ups designed in Figma are included in the Scope Document appendix as illustrative references only. They are not committed screen designs and may differ from the final implementation.

6.2 Software interfaces

ID	System	Version	Interface Description
SI-1	Judge0 CE	API v1 (self-hosted)	REST API. POST /submissions to submit code with language_id, source_code, stdin, cpu_time_limit, memory_limit. GET /submissions/{token} to poll results.
SI-2	Ollama	Latest stable	REST API at http://ollama:11434. POST /api/generate with model: llama3.1:8b-instruct, prompt: structured rubric prompt, stream: false. Returns JSON response field.
SI-3	Redis 7.2+	7.2+	Celery message broker on port 6379 (DB 0). Explanation cache on DB 1 using key pattern explanation:{submissionId}:{type}. TTL: 30 days.
SI-4	PostgreSQL	15+	Primary data store. Connected via SQLAlchemy 2.x async ORM using asyncpg driver. Schema migrations via Alembic. Connection pool: 10 min, 20 max connections.
SI-5	sentence-transformers	3.x	Local Python library. Model: all-MiniLM-L6-v2. Used for: (a) submission embedding generation (M-05), (b) question embedding for RAG retrieval (M-09).
SI-6	LangChain	0.2+	Python library for RAG pipeline in M-09. Uses document loaders (PDF, DOCX, PPTX), RecursiveCharacterTextSplitter, FAISS vector store, and RetrievalQA chain with Ollama LLM.

SI-7	FastAPI OpenAPI	Auto-generated	Backend auto-generates OpenAPI 3.0 schema at /docs (Swagger UI) and /redoc. Used for API contract verification and testing.
------	--------------------	----------------	---

6.3 Hardware interfaces

Compatibility Requirements

- HI-1: The Ollama container requires access to the host GPU for accelerated LLM inference via the docker compose --gpus all flag. CPU-only fallback is supported with an approximately 5x increase in grading latency.
- HI-2: The Flutter mobile application interfaces with device biometric hardware (fingerprint sensor or Face ID) via the Flutter local_auth package. The application falls back to password authentication on devices without enrolled biometrics.
- HI-3: The server requires minimum 16 GB RAM to run all Docker Compose services simultaneously: FastAPI backend, Celery workers (x2), Redis, PostgreSQL, Ollama, and Judge0.

Operating Requirements

- HI-4: The Flutter mobile application interfaces with the device operating system AppLifecycleState change notification system to detect foreground/background transitions for proctoring. This is a pure software interface with no hardware dependency beyond the operating system scheduler.
- HI-5: All Docker containers shall run on an isolated internal network with only the NGINX reverse proxy exposing external ports 80 and 443.

6.4 Communications interfaces

- CI-1: All HTTP communication between web clients and the FastAPI backend shall use HTTPS with TLS 1.3 in production. HTTP shall be disabled on the NGINX reverse proxy.
- CI-2: Real-time proctoring violation events shall be transmitted from the Flutter mobile application to the FastAPI backend via WebSocket Secure (WSS). The WebSocket connection

shall be established when the student opens a proctored examination and maintained until submission or timeout.

- CI-3: In-platform grade release and assignment notifications shall be delivered via Server-Sent Events (SSE) from the FastAPI backend to connected web portal clients.
- CI-4: Email notifications for account verification, password reset, and optional grade alerts shall be sent via SMTP using the university email server. SMTP host, port, TLS mode, sender address, and credentials shall be configurable via Docker Compose environment variables with no hardcoded values.

7. Requirements Traceability Matrix

The Requirements Traceability Matrix (RTM) maps every functional requirement to: (1) the business objective it satisfies, (2) the module it belongs to, and (3) the test case that will verify it. This ensures no requirement exists without a business justification and no requirement is left untested.

Req. ID	Module	Business Objective	Source	Rationale	Priorty	Test Case ID
FR-M01	M-01: Authentication and User Management	BO-10: Secure role-controlled deployment	Supervisor; STMU observation	Foundation for all module security and audit trail	High	TC-AUTH-01
FR-M02	M-02: Course and Assignment Management	BO-1: Grading pipeline foundation; BO-3: Rubric quality drives IIR	Supervisor; observation session	Rubric quality directly determines AI grading accuracy	High	TC-COURSE-01
FR-M03	M-03: AI Grading and Evaluation	BO-1: 75% grading time reduction; BO-2:	Supervisor; BO-1, BO-2, BO-3	Eliminates the documented 12-18	High	TC-GRADE-01

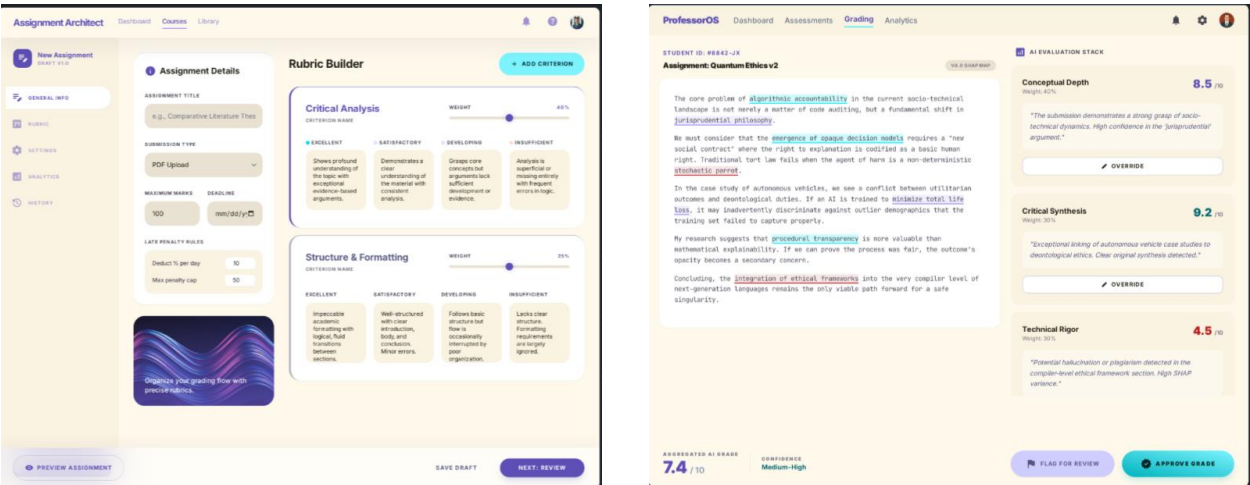
		60-second feedback; BO-3: 85% IIR		hr/week grading burden at STMU		
FR-M04	M-04: Explainable AI (XAI) Processing	BO-4: LIME/SHAP transparency for students and professors	Co-Supervisor; BO-4	Addresses trust barrier: makes AI grading rationale auditable	High	TC-XAI-01
FR-M05	M-05: Student Submission Portal	BO-1: Unified submission; BO-2: Feedback delivery; BO-4: XAI viewing	Supervisor; student interviews	Single portal for entire student assessment lifecycle	High	TC-SUB-01
FR-M06	M-06: Analytics Dashboard	BO-9: Real-time at-risk identification and course performance	Supervisor; BO-9	Enables early academic intervention; no structured analytics mechanism exists at STMU	High	TC-DASH-01
FR-M07	M-07: Smart Proctoring	BO-6: 60% reduction in single-device cheating	Co-Supervisor; STMU admin privacy constraint	App-switch detection without cameras satisfies STMU privacy policy	High	TC-PROC-01
FR-M08	M-08: Code Editor and Compiler	BO-5: Sandboxed in-browser code	Co-Supervisor; BO-5	Eliminates manual code-environment setup for	High	TC-CODE-01

		execution for 12 languages		programming assignments		
FR-M9	M-09: AI Teaching Assistant	BO-8: 24/7 course-specific RAG-grounded student support	Supervisor; student interviews	Reduces out-of-hours response delays from days to seconds	Medium	TC-CHAT-01
REL-1	Reliability — Inter-Rater Reliability	BO-3: Consistent rubric-based grading above 85% IIR	Supervisor; BO-3	Consistency verified via paired test across 50 submissions; 3-mark TA swing observed at STMU	High	TC-IRR-01
NFR-PERF-1	Performance — 60s Feedback Under Load	BO-2: 60-second feedback delivery for 180 concurrent students	BO-2	Concurrent load target derived from max STMU cohort size	High	TC-PERF-01
NFR-SEC-1 to SEC-6	Security (M-01 through M-09 cross-cutting)	BO-10: Secure institutional deployment	BO-10; IEEE 830	JWT, Argon2id, RBAC, input sanitisation, sandboxed execution	High	TC-SEC-01
NFR-MNT-1	Maintainability — Docker Deployment	BO-10: Single-command deployment	BO-10	Eliminates IT setup overhead for university server deployment via single-command docker compose up -build -d	High	TC-DEPLOY-01

NFR-MNT-2	Maintainability CI/CD Pipeline	—	BO-10: Automated testing on every commit	BO-10	Prevents regression; CI blocks deployment on any failing test	High	TC-DEPLOY-02
-----------	--------------------------------	---	--	-------	---	------	--------------

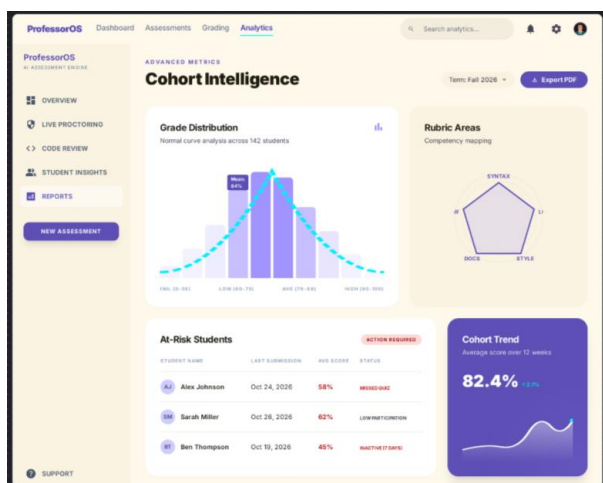
8. Storyboarding

High-fidelity wireframes for the nine core modules of ProfessorOS are designed using Figma. The storyboards below illustrate the key screens, user flows, and interface states across the platform.

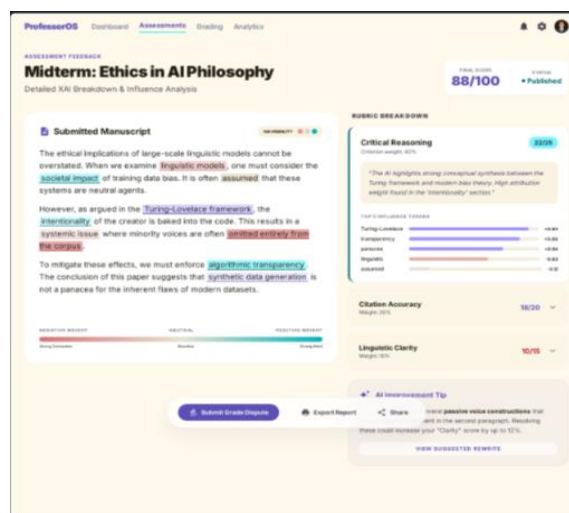


1- The Rubric Builder screen displays a two-panel assignment creation wizard. The left panel contains assignment metadata fields and the right panel shows the Rubric Builder with criterion rows. A Preview Assignment button launches a student-view simulation overlay.

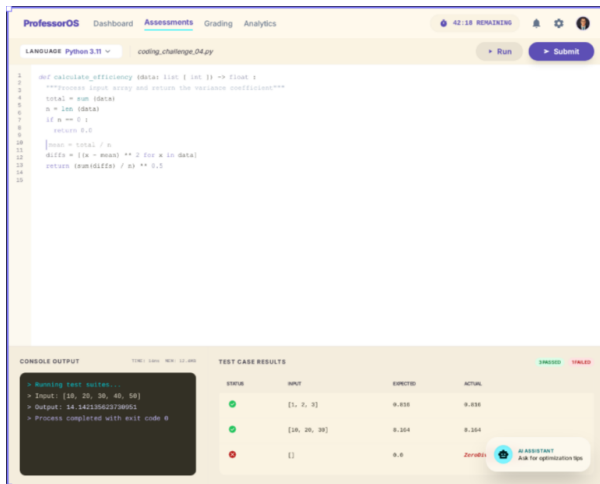
2- A split-panel grading interface. The left panel shows the student submission with a SHAP highlight overlay. The right panel shows criterion-level cards with AI scores, feedback, and override controls. Approve Grade and Flag for Review buttons appear at the bottom.



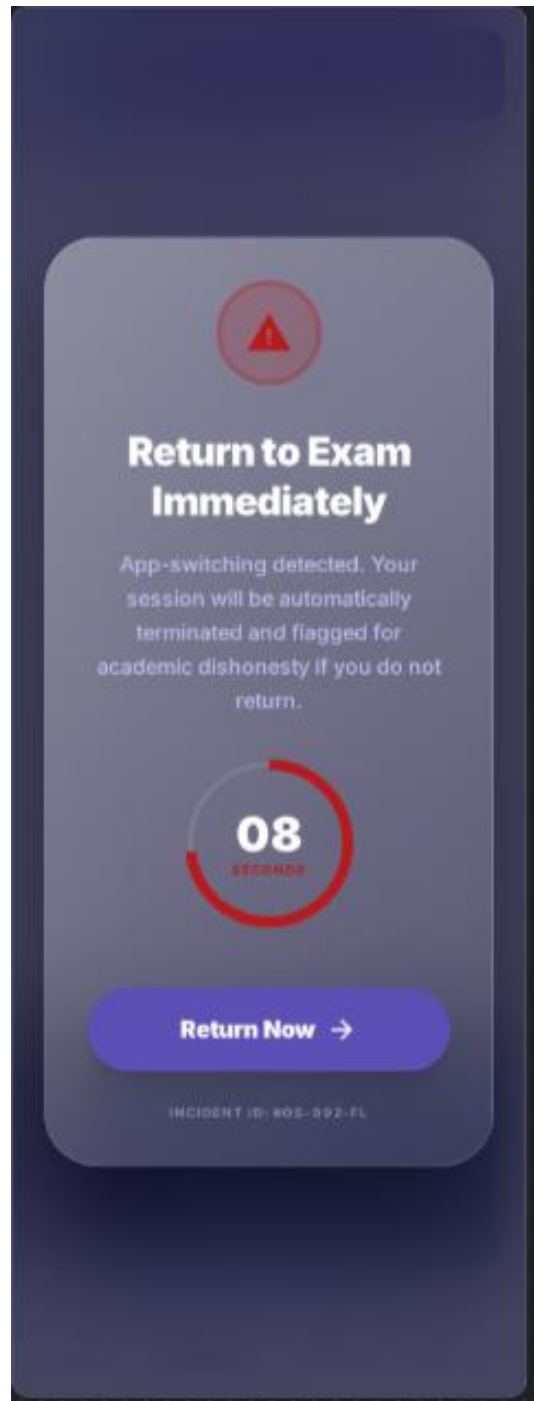
3- A four-panel data dashboard showing a grade distribution histogram, a rubric criterion radar chart, an at-risk students table, and a cohort score time-series line chart across all assignments in the semester.



4- The student-facing feedback view shows submitted text with LIME color-coded highlights on the left. Each rubric criterion on the right shows earned score, AI feedback, and a bar chart of the top five tokens that influenced that criterion score.



5- A full-screen Monaco Editor interface with a language selector, Run and Submit buttons, and a time indicator. Below the editor, the output panel shows console results on the left and test case pass/fail results on the right.



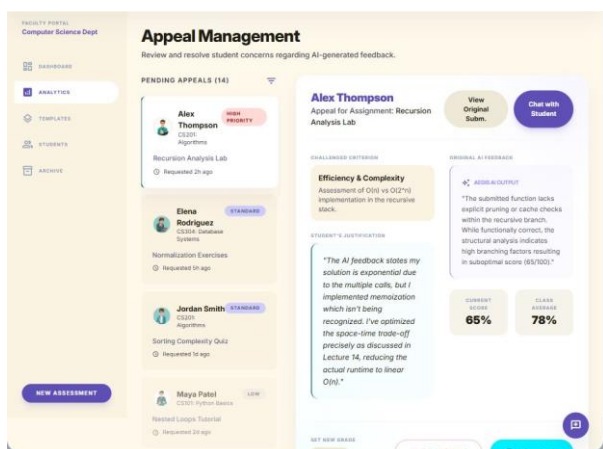
6- The Flutter mobile exam screen showing the Focus Mode Active banner with violation count and remaining time. A full-screen

overlay with a red countdown timer and Return Now button appears when the student switches applications.

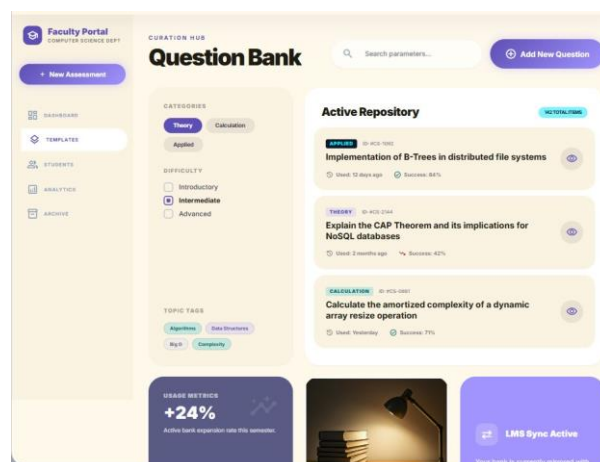
7- Step 4 of 5 in the exam creation wizard.

The Availability and Timeline panel lets professors set open/close dates and times with timezone support. A visual window preview bar shows the exam window duration. A Duration card below defines the time allowed once a student begins.

8- The Enrollment Matrix screen shows active courses for the semester with enrollment progress bars. The Enrollment Manager below lets faculty drag students from the master roster into specific course sections. A destination panel shows available slots and accepts multi-select via CMD/CTRL.



9- A two-panel appeal review interface. The left panel lists pending student appeals with priority tags and timestamps. The right panel shows the selected appeal in detail, displaying the challenged criterion, the original AEGIS AI output, the student justification, current score versus class average, and resolution controls to reject or adjust the grade.



10- The Question Bank screen provides a filterable repository of assessment questions. Filters by category, difficulty, and topic tags appear on the left. The active repository on the right shows each question with type badge, ID, last usage date, and success rate. Usage metrics and LMS sync status are displayed at the bottom.

9. References

- Ribeiro, M. T., Singh, S., and Guestrin, C. (2016). Why Should I Trust You?: Explaining the Predictions of Any Classifier. Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 1135-1144.
- Lundberg, S. M., and Lee, S. I. (2017). A Unified Approach to Interpreting Model Predictions. Advances in Neural Information Processing Systems, vol. 30.
- Reimers, N., and Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Proceedings of EMNLP, pp. 3982-3992.
- Shermis, M. D., and Burstein, J. (2013). Handbook of Automated Essay Evaluation. New York: Routledge.
- Meta AI. (2024). Llama 3 Technical Report. Meta Platforms Inc. Available at: <https://ai.meta.com/research/publications/>
- Ramesh, A., and Pradeep, P. (2022). Automated Assessment in Education: A Literature Review. IEEE Transactions on Learning Technologies, vol. 15, no. 3, pp. 301-315.
- Judge0. (2024). Judge0 CE: Open-Source Online Code Execution System. GitHub. Available at: <https://github.com/judge0/judge0>
- Higher Education Commission (HEC) Pakistan. (2023). Annual Report 2022-2023: Higher Education Statistics. HEC, Islamabad.
- Wang, W., and Scott, D. (2016). Automatic Essay Scoring: A Survey. Natural Language Engineering, vol. 22, no. 4, pp. 537-571.
- Google. (2024). Flutter: Build apps for any screen. Available at: <https://flutter.dev>
- Hugging Face. (2024). sentence-transformers: Multilingual Sentence Embeddings using BERT. GitHub. Available at: <https://github.com/UKPLab/sentence-transformers>
- Ollama. (2024). Ollama: Get up and running with large language models locally. GitHub. Available at: <https://github.com/ollama/ollama>